UNIVERSIDAD DEL VALLE DE GUATEMALA

Programación Paralela y Distribuida

Sección 10

Ing. Kimberly Barrera



Proyecto No. 2

Adrián González - 23152

Fernando Mendoza - 19644

Hansel López - 19026

Guatemala, 23 de octubre del 2026

Inciso 1. Cifrado y descifrado de archivos con AES-128

Se implementó el programa aes_archivo.c para leer un archivo de texto y cifrar su contenido con AES-128-CBC mediante una clave numérica recibida como parámetro. La función make_key() convierte el número a un arreglo de 16 bytes: los primeros ocho permanecen en cero y los últimos ocho contienen el valor en formato big-endian.

Para procesar textos de longitud variable se utiliza relleno PKCS#7, administrado por OpenSSL. En cada cifrado se genera un vector de inicialización aleatorio de 16 bytes, que se guarda al inicio del archivo junto con el contenido cifrado. Durante el descifrado se recupera ese IV y se utiliza la misma clave para reconstruir el texto original.

```
adridan@ADRIDAN:/mnt/c/Users/adria/Downloads$ ./aes_archivo cifrar    -k 45643 -i mensaje.txt -o salida.bin
Clave: 45643
Clave AES-128 (16 bytes):
00000000000000000000000000000b24b
Texto original (32 bytes):
"Esta es una prueba de proyecto 2"
IV (16 bytes):
b3d812b82ad908914c959cf91bf2ac54
Texto cifrado (48 bytes):
09c2704f86080cd8487b5b7b58878cda56a8477829c7b3a309928d68bafd8713
684bcc0388499990051998eafac24236
Archivo cifrado guardado en: salida.bin
```

Figura 1. Compilación y demostración del cifrado.

Los modos cifrar y descifrar permiten guardar y recuperar el contenido desde disco. La comparación con cmp verifica que el archivo recuperado sea idéntico al original.

```
adridan@ADRIDAN:/mnt/c/Users/adria/Downloads$ ./aes_archivo descifrar -k 45643 -i salida.bin -o salida.txt
Clave: 45643
Clave AES-128 (16 bytes):
00000000000000000000000000000b24b
Texto cifrado (48 bytes):
09c2704f86080cd8487b5b7b58878cda56a8477829c7b3a309928d68bafd8713
684bcc0388499990051998eafac24236
Texto recuperado (32 bytes):
"Esta es una prueba de proyecto 2"
Archivo descifrado guardado en: salida.txt
```

Figura 2. Cifrado y descifrado mediante archivos independientes.

El mensaje “Esta es una prueba de proyecto 2”, sin salto de línea, ocupa 32 bytes. PKCS#7 agrega un bloque completo de 16 bytes, por lo que el contenido cifrado ocupa 48 bytes. Al incluir el IV, el archivo resultante tiene 64 bytes. El descifrado elimina el relleno y recupera los 32 bytes originales.

Inciso 2. Búsqueda paralela de la clave y pruebas de tiempo

Usamos la versión paralela base (reparto por bloques contiguos) corriendo siempre con 4 procesos (-np 4). El proceso 0 cifra el texto “Esta es una prueba de proyecto 2” y reparte el cifrado y el IV a todos; cada proceso recorre su cuarto del espacio de claves y verifica cada intento buscando la frase “es una prueba de” en el texto descifrado. Medimos tres claves:

```
adridan@ADRIDAN:/mnt/c/Users/adria/Downloads$ mpicc -std=c11 -O2 -Wall -Wextra busqueda_clave_aes_mpi.c -o busqueda_clave_aes_mpi -lcrypto
adridan@ADRIDAN:/mnt/c/Users/adria/Downloads$ mpirun -np 4 ./busqueda_clave_aes_mpi -k 123456           -b 24
Estrategia: base (bloques contiguos)
Procesos MPI: 4
Rango explorado: 2^24 = 16777216 candidatas
Claves por proceso: ~4194304
Clave encontrada: 123456
Tiempo: 0.161858 segundos
adridan@ADRIDAN:/mnt/c/Users/adria/Downloads$ mpirun -np 4 ./busqueda_clave_aes_mpi -k 18014398509481984 -b 56
Estrategia: base (bloques contiguos)
Procesos MPI: 4
Rango explorado: 2^56 = 72057594037927936 candidatas
Claves por proceso: ~18014398509481984
Clave encontrada: 18014398509481984
Tiempo: 0.006674 segundos
adridan@ADRIDAN:/mnt/c/Users/adria/Downloads$ mpirun -np 4 ./busqueda_clave_aes_mpi -k 18014398509481985 -b 56
Estrategia: base (bloques contiguos)
Procesos MPI: 4
Rango explorado: 2^56 = 72057594037927936 candidatas
Claves por proceso: ~18014398509481984
Clave encontrada: 18014398509481985
Tiempo: 0.005140 segundos
adridan@ADRIDAN:/mnt/c/Users/adria/Downloads$
```

Figura 3. Ejecución de las tres pruebas de búsqueda con 4 procesos

Tiempos reales:

a) clave 123456 → 0.161858 s

b) clave 18014398509481984 → 0.006674 s

c) clave 18014398509481985 → 0.005140 s

d) ¿Por qué varía el tiempo según la clave?

Lo interesante es que la clave más grande (b y c, del orden de 10^{16}) se encontró más rápido que la clave chica (a, 123456). Esto pasa porque con el reparto por bloques contiguos el tiempo no depende de qué tan grande es el número de la clave, sino de qué tan adentro del bloque de su proceso cae.

En la prueba a), la clave 123456 está dentro del bloque del proceso 0, que arranca en 0, así que ese proceso tiene que probar 123456 claves hasta llegar a ella mientras los otros tres buscan en zonas donde no está. Por eso tarda más.
En las pruebas b) y c), la clave $2^{56}/4$ cae justo en la frontera donde arranca el bloque del proceso 1 (cada proceso recibe $2^{56}/4$ claves). El proceso 1 la encuentra en su primer intento, y la de c) en el segundo. Por eso son casi instantáneas, aunque el número sea gigante.

Inciso 3. Propuesta para mejorar la búsqueda de claves

Estrategia propuesta: distribución cíclica (intercalada) de las claves

Repartimos el espacio de claves en 4 bloques grandes y seguidos: el proceso 0 se queda con el primer cuarto, el 1 con el segundo, y así. El problema es que la clave puede caer muy adentro del bloque de un solo proceso, y ese proceso termina haciendo casi todo el trabajo mientras los otros tres ya acabaron su parte y se quedan sin hacer nada.

Nuestra propuesta es cambiar ese reparto por uno cíclico, tipo por turnos. En vez de darle a cada proceso un bloque corrido, las claves se van repartiendo intercaladas: el proceso 0 prueba la clave 0, 4, 8, 12…; el proceso 1 prueba la 1, 5, 9, 13…; el proceso 2 la 2, 6, 10…; y el proceso 3 la 3, 7, 11… Dicho simple, en lugar de partir el espacio en cuatro pedazos grandes, los cuatro procesos avanzan juntos por el espacio como cuatro carriles paralelos, cada uno agarrando una de cada cuatro claves.

Distribución del trabajo entre los procesos

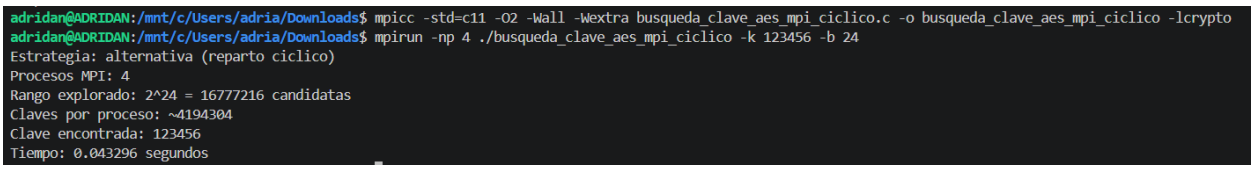
Cada proceso, según su rank (0 a 3), arranca en su propio número y avanza de 4 en 4 (en general, de “cantidad de procesos” en adelante). Así, con 4 procesos, entre los cuatro cubren cualquier tramo del espacio de claves al mismo tiempo, sin que a nadie le toque un bloque aislado. La comunicación es igual: el proceso 0 cifra el mensaje una sola vez y reparte el texto cifrado y el IV a todos; cada quien prueba sus claves descifrando y revisando si aparece la frase “es una prueba de”; y en cuanto alguno encuentra la clave, avisa a los demás para que todos paren y no sigan buscando de por gusto.

Justificación:

La ganancia principal es el balance de carga. Con el reparto cíclico, esa misma zona la recorren los cuatro procesos a la vez, así que una clave que está profunda dentro de un

bloque se encuentra aproximadamente cuatro veces más rápido, porque el trabajo de llegar hasta ella se reparte entre todos en lugar de recaer en uno. Además, ningún proceso se queda ocioso antes de tiempo: todos siguen aportando hasta que la clave aparece.

Consideramos que esta estrategia no gana siempre. Cuando la clave cae justo al inicio del bloque de un proceso (como pasa con $2^{56}/4$, que en el reparto por bloques el proceso 1 encuentra casi de inmediato), el reparto por bloques puede ser más rápido. Esa diferencia es justamente lo interesante para el análisis: el reparto cíclico mejora el caso en que la clave está metida adentro de un bloque y empareja la carga entre procesos, mientras que el reparto por bloques aprovecha mejor las claves que caen cerca de las fronteras. Esa es nuestra propuesta y la ventaja y limitación de cada una.



Comparación de las dos estrategias con las mismas claves del inciso 2

Corrimos las dos versiones (bloques contiguos y reparto cíclico) con las tres claves del inciso 2, siempre con 4 procesos:

Prueba	**Clave**	**Estrategia base (bloques)**	**Estrategia cíclica**
a)	123456	0.161858 s	0.100003 s
b)	18014398509481984 ($2^{56}/4$)	0.006674 s	~305 años (estimado por IA)
c)	18014398509481985 ($2^{56}/4 + 1$)	0.005140 s	~305 años (estimado por IA)

Para b) y c) el cíclico no llega a terminar: medimos la velocidad real del programa (~467,500 claves por segundo por proceso) y con ella el proceso al que le toca la clave tendría que hacer unas 2^{52} iteraciones, lo que da un estimado de cientos de años. Por eso no se corrieron hasta el final, solo se calculó el tiempo.

Análisis, ventajas y limitaciones

En la prueba a) ganó el reparto cíclico: la clave 123456 está en la parte baja del espacio, y como los cuatro procesos la recorren juntos, se reparte el trabajo de llegar hasta ella y termina más rápido que la base, donde el proceso 0 la busca prácticamente solo.

En b) y c) pasó lo contrario, y de forma extrema. Esas claves caen justo en la frontera donde arranca el bloque de un proceso, así que la estrategia base las encuentra casi de inmediato. En cambio el cíclico reparte esas mismas claves muy adentro de la secuencia de un proceso (alrededor de la iteración 2^{52}), y por eso se vuelve inviable.

La conclusión es que ninguna estrategia es mejor siempre; depende de dónde esté la clave:

Reparto por bloques (base).

Ventaja: encuentra muy rápido las claves que caen cerca del inicio de algún bloque, como las fronteras 0, $2^{56}/4$, $2 \cdot 2^{56}/4$, etc.

Limitación: si la clave cae profundo dentro de un bloque, ese proceso carga casi toda la búsqueda mientras los otros tres terminan temprano y se quedan ociosos (mal balance de carga).

Reparto cíclico (alternativo).

Ventaja: mejor balance de carga, los cuatro procesos avanzan juntos y una clave en la zona baja del espacio se encuentra más rápido.

Limitación: pierde la ventaja de las fronteras; una clave que en bloques caía en el arranque de un proceso, en cíclico queda hasta el fondo de la secuencia de ese proceso, y el tiempo se dispara.

Inciso 4 Estrategia Alternativa MPI

Este apartado describe la estrategia alternativa de distribución cíclica propuesta para la búsqueda paralela de claves. Su propósito es repartir el trabajo de forma uniforme entre los procesos MPI y coordinar la finalización cuando se identifica la clave correcta.

Funcionamiento de la estrategia

La estrategia alternativa asigna las claves de forma cíclica o intercalada. En lugar de entregar un bloque continuo a cada proceso, cada proceso recibe una de cada n claves, donde n es la cantidad total de procesos MPI. De esta forma, el proceso con rango r prueba las claves r, r + n, r + 2n y así sucesivamente.

Con cuatro procesos, la distribución se muestra a continuación. Cada clave se asigna a un único proceso, por lo que no se repiten ni se omiten candidatas.

Proceso MPI	Claves asignadas
Proceso 0	0, 4, 8, 12, 16, ...
Proceso 1	1, 5, 9, 13, 17, ...
Proceso 2	2, 6, 10, 14, 18, ...
Proceso 3	3, 7, 11, 15, 19, ...

Esquema de flujo de la búsqueda

1. Proceso 0 cifra el mensaje y genera el vector de inicialización IV.
2. El proceso 0 distribuye el texto cifrado y el IV a todos los procesos mediante MPI_Bcast.
3. Cada proceso obtiene su rango y prueba las claves que le corresponden de forma intercalada.
4. Cada intento descifra el texto y verifica si contiene la frase de búsqueda es una prueba de.
5. Al finalizar una ronda, los procesos usan MPI_Allreduce para verificar si alguien encontró la clave.
6. Si existe una clave encontrada, todos finalizan; de lo contrario, comienza una nueva ronda.

Pseudocódigo

```
Proceso 0:
    leer la clave secreta y la cantidad de bits
    compartir parámetros con MPI_Bcast
    generar IV aleatorio
    cifrar el mensaje con AES 128 CBC
    compartir IV y texto cifrado con MPI_Bcast

Todos los procesos:
    stride = cantidad_procesos
    clave = rank
    clave_local = NO_ENCONTRADA
    sincronizar con MPI_Barrier

    mientras no se cumpla la condición de parada:
        probar hasta 4096 claves, avanzando clave += stride
        si una clave descifra el texto correctamente:
            guardar clave_local

        found_any = MPI_Allreduce(clave_local, MPI_LOR)
        active_any = MPI_Allreduce(trabajo_pendiente, MPI_LOR)
        detenerse si found_any es verdadero o active_any es falso

    clave_global = MPI_Allreduce(clave_local, MPI_MIN)
    tiempo_global = MPI_Reduce(tiempo_local, MPI_MAX)
    reportar la clave encontrada y finalizar MPI
```

Comunicación y finalización

La comunicación se realiza en dos etapas. Primero, MPI_Bcast permite que todos los procesos reciban el mismo IV y texto cifrado. La clave secreta no se transmite, pues cada proceso debe encontrarla por fuerza bruta. Posteriormente, MPI_Allreduce permite compartir el resultado de cada ronda: si un proceso encuentra una clave válida, los demás reciben la información y terminan sin continuar con intentos innecesarios.

La ventaja principal es el balance de carga, ya que todos los procesos progresan sobre las mismas zonas del espacio de búsqueda al mismo tiempo. Como limitación, existe un costo de sincronización por las comunicaciones colectivas; además, la estrategia por bloques puede resultar favorable cuando la clave se ubica al inicio de un bloque asignado.